



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

FUTURE BACKEND INTEGRATION

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	FUTURE BACKEND INTEGRATION
Version	v1.0.0
Date	02 July 2026

How to Connect FastAPI to This React Frontend

1. Overview

The frontend is **backend-ready**. Every API call is already written in the service layer. The Axios client has JWT interceptors. The authentication flow is structured. When the FastAPI backend is ready, integration requires:

1. Set the API URL in `.env`
2. Remove mock data from `AuthContext.jsx`
3. Connect React Query `useQuery` hooks in pages

No component redesign. No routing changes. No layout changes.

2. Step-by-Step Integration Checklist

Step 1 — Set Environment Variable

```
# .env
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

Step 2 — Connect Authentication

Open `src/contexts/AuthContext.jsx` and update the `login` function:

```
// BEFORE (mock):
const data = {
  user: { id: '1', name: 'Demo User', email: credentials.email, role: 'patient' },
  accessToken: 'mock_access_token',
  refreshToken: 'mock_refresh_token',
}

// AFTER (real API):
import authService from '../services/authService'
const { data } = await authService.login(credentials)
```

Do the same for `register`, `verifyOtp`, `resendOtp`, `forgotPassword`, `resetPassword`.

Step 3 — Enable Token Refresh (optional but recommended)

In `axiosClient.js`, replace the 401 redirect with a refresh flow:

```
// When 401 occurs:
// 1. Try: POST /auth/refresh-token with refreshToken
// 2. If success: update ACCESS_TOKEN, retry original request
// 3. If failure: clear session + redirect to /session-expired
```

Step 4 — Enable Session Guard

In `App.jsx`:

```
// Change:
```

```

useSessionGuard({ enabled: false })

// To:
const { logout } = useAuth()
useSessionGuard({ enabled: true, onTimeout: logout, timeoutMs: 30 * 60 * 1000 })

```

Step 5 — Connect Pages with React Query

Example for MedicineDetailsPage:

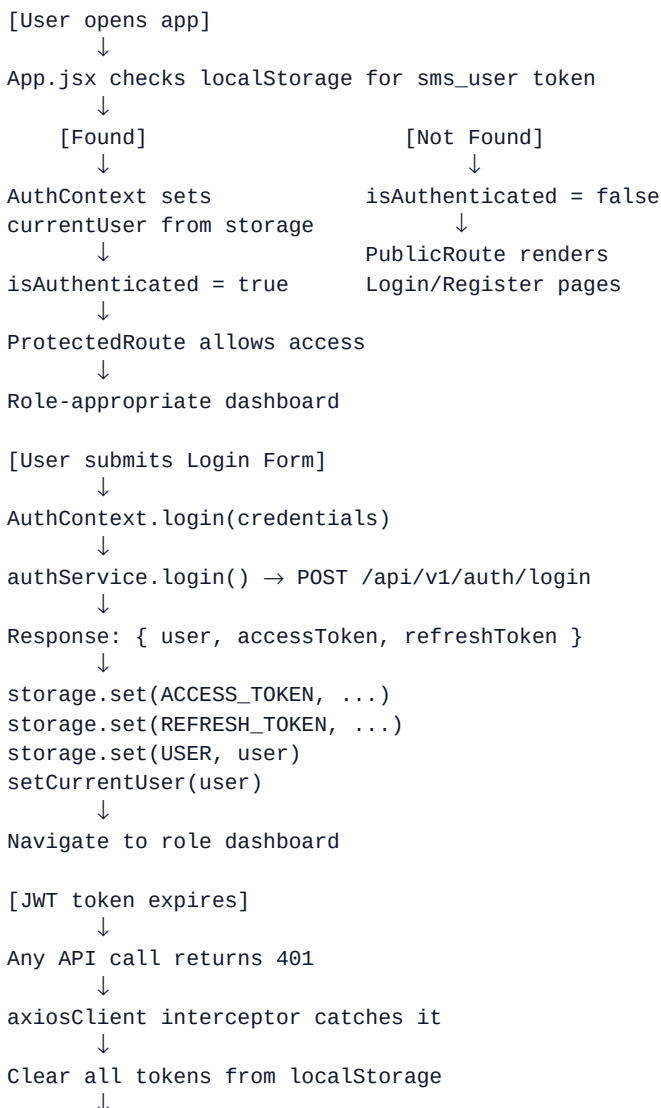
```

import { useQuery } from '@tanstack/react-query'
import medicineService from '../../services/medicineService'

const { data: medicine, isLoading, isError } = useQuery({
  queryKey: ['medicine', id],
  queryFn: () => medicineService.getById(id).then(res => res.data),
})

```

3. Authentication Flow Diagram



```

window.location.href = '/session-expired'
↓
User sees SessionExpiredPage
→ "Go to Login" button

```

4. Medicine Search Flow

```

[User types in search box]
↓
useDebounce(query, 400ms) – wait for user to pause
↓
React Query fires: medicineService.search({ q: debouncedQuery })
↓
GET /api/v1/medicines/search?q=parace&limit=10
↓
FastAPI queries MongoDB/PostgreSQL
Returns: [{ id, name, genericName, price, availability, ... }]
↓
React Query caches result
↓
SearchResultsPage renders SearchResultCard list
↓
User clicks "View Details"
↓
navigate('/medicine/123')
↓
MedicineDetailsPage: medicineService.getById('123')
↓
GET /api/v1/medicines/123
Returns: { full medicine object }
↓
Page renders with complete medicine information

```

5. Generic Recommendation Flow

```

[User on MedicineDetailsPage clicks "View Generic Alternative"]
↓
navigate('/medicine/123/generic')
↓
GenericRecommendationPage loads
↓
medicineService.getAlternatives('123')
↓
GET /api/v1/medicines/alternatives/123
FastAPI queries generic mapping table
Returns: [{
  id, name, genericName, manufacturer,
  price, mrp, savings, isJanAushadhi,
  composition, similarityScore
}]
↓
Page renders:
- Branded medicine summary
- Generics list sorted by price
- Savings calculator
- "Find Nearby Pharmacy" button

```

6. Nearby Pharmacy Flow

```

[User clicks "Find Nearby Pharmacies"]
↓
Browser asks for location permission
↓
navigator.geolocation.getCurrentPosition()
Returns: { lat, lng }
↓
pharmacyService.getNearby({ lat, lng, radius: 5, medicineId: '123' })
↓
GET /api/v1/pharmacies/nearby?lat=19.07&lng=72.87&radius=5&medicineId=123
FastAPI queries pharmacy database with geospatial index
Returns: [{
  id, name, address, lat, lng,
  distance, isOpen, phone,
  hasStock: true
}]
↓
React Leaflet renders markers on map
PharmacyListPanel renders sorted list
↓
User clicks "Get Directions"
↓
Opens Google Maps: https://maps.google.com/?q={lat},{lng}

```

7. Inventory Management Flow

```

[Pharmacist views InventoryPage]
↓
inventoryService.getAll({ page: 1, limit: 10, search: '' })
↓
GET /api/v1/inventory?page=1&limit=10
Returns: { items: [...], total, page, totalPages }
↓
Table renders with pagination

[Pharmacist clicks "Add Medicine"]
↓
Navigate to /pharmacy/inventory/add
MedicineFormPage (add mode – no :id in URL)
↓
User fills form: name, batch, quantity, price, expiry
↓
inventoryService.create(formData)
↓
POST /api/v1/inventory
Returns: { created stock item }
↓
React Query invalidates ['inventory'] cache
↓
InventoryPage auto-refreshes with new item

```

8. Admin Flow

```

[Admin views AdminUsers]
  ↓
userService.getAllUsers({ page: 1, role: 'patient' })
  ↓
GET /api/v1/users?page=1&role=patient
Returns: { users: [...], total }
  ↓
AdminTable renders

[Admin changes user role]
  ↓
PUT /api/v1/users/:id
Body: { role: 'pharmacist' }
  ↓
React Query invalidates ['users'] cache
Table refreshes

```

9. Notification Flow

```

[User opens NotificationsPage]
  ↓
GET /api/v1/users/me/notifications
Returns: [{ id, type, title, message, time, isRead }]
  ↓
NotificationCard list renders

[User marks one as read]
  ↓
PATCH /api/v1/users/me/notifications/:id/read
  ↓
React Query invalidates ['notifications'] cache
Badge count updates in TopBar

```

10. FastAPI Backend Expected Structure

The frontend expects these FastAPI endpoint groups:

```

# FastAPI router structure (backend developer reference)
app.include_router(auth_router,      prefix="/api/v1/auth")
app.include_router(user_router,      prefix="/api/v1/users")
app.include_router(medicine_router,  prefix="/api/v1/medicines")
app.include_router(pharmacy_router,  prefix="/api/v1/pharmacies")
app.include_router(inventory_router, prefix="/api/v1/inventory")
app.include_router(analytics_router, prefix="/api/v1/analytics")

```

CORS configuration required:

```

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"], # Vite dev server
    allow_credentials=True,

```

```

    allow_methods=["*"],
    allow_headers=["*"],
)

```

11. Expected Response Format

The frontend error handling expects this FastAPI response shape:

Success:

```

{
  "success": true,
  "data": { ... },
  "message": "Success"
}

```

Error:

```

{
  "success": false,
  "message": "Invalid credentials",
  "error": "INVALID_CREDENTIALS"
}

```

Paginated:

```

{
  "data": [...],
  "total": 150,
  "page": 1,
  "totalPages": 15,
  "limit": 10
}

```

The `AuthContext` error handling reads:

```
const msg = err?.response?.data?.message ?? 'Login failed. Please try again.'
```

This matches the FastAPI error response shape above.